SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 15

Name of Student : Kesoju Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task 1 – Student Attendance API
Create a RESTful API for student attendance tracking.
Instructions:
• Endpoints required:
o GET /attendance → View attendance records
o POST /attendance → Mark attendance
o PUT /attendance/{id} → Update attendance record
o DELETE /attendance/{id} → Remove attendance entry
• Ensure JSON-based request and response handling.
Expected Output:
• API for maintaining and updating attendance records.

Prompt:-
Develop a simple **Student Attendance REST API** using **Node.js and Express**. The API should allow users to retrieve attendance records, add new attendance entries, update an existing record using its ID, and delete a record when required. Store the data temporarily in an in-memory array and return responses in JSON format with basic validation and error messages.
Code:-

SR UNIVERSITY

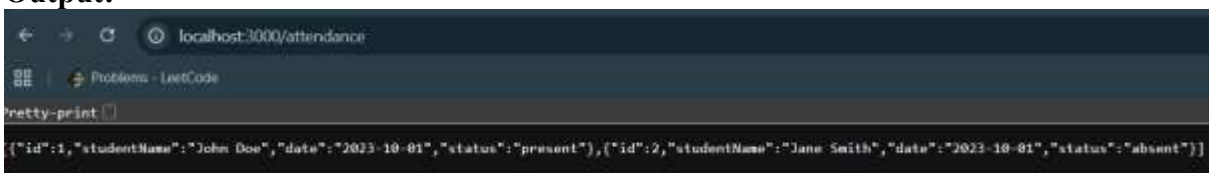
Task-1.js > ...

```javascript
//Develop a simple Student Attendance REST API using Node.js and Express. The API should allow users to
const express = require('express');
const app = express();
const PORT = 3000;
app.use(express.json());

// In-memory array to store attendance records
let attendanceRecords = [
    // Example record
    {
        id: 1,
        studentName: 'John Doe',
        date: '2023-10-01',
        status: 'present'
    },
    {
        id: 2,
        studentName: 'Jane Smith',
        date: '2023-10-01',
        status: 'absent'
    }
];
let currentId = 2; // To keep track of the next ID to assign
// GET /attendance - View all attendance records
app.get('/attendance', (req, res) => {
    res.json(attendanceRecords);
});
// POST /attendance - Mark attendance
app.post('/attendance', (req, res) => {
    const { studentName, date, status } = req.body;
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    const newRecord = {
        id: currentId++,
        studentName,
        date,
        status
    };
    attendanceRecords.push(newRecord);
    res.status(201).json(newRecord);
});
// PUT /attendance/:id - Update a record
app.put('/attendance/:id', (req, res) => {
    const { id } = req.params;
```

```
app.post('/attendance', (req, res) => {
    const newRecord = {
    };
    attendanceRecords.push(newRecord);
    res.status(201).json(newRecord);
});
// PUT /attendance/:id - Update a record
app.put('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const { studentName, date, status } = req.body;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    attendanceRecords[recordIndex] = { id: parseInt(id), studentName, date, status };
    res.json(attendanceRecords[recordIndex]);
});
// DELETE /attendance/:id - Remove a record
app.delete('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    const deletedRecord = attendanceRecords.splice(recordIndex, 1);
    res.json(deletedRecord[0]);
});
// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on port http://localhost:${PORT}`);
});
```

Output:-



Justification:-

This implementation creates a REST API that manages student attendance records through different HTTP methods. It allows users to add, view, update, and remove attendance entries. The system stores records in memory, which is suitable for testing and demonstration purposes. Error handling ensures that invalid operations are handled properly, and JSON responses make it easy for client applications to interact with the API.

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

- **Endpoints required:**
  - **GET /inventory → List all items**
  - **POST /inventory → Add new item**
  - **PUT /inventory/{id} → Update stock quantity**
  - **DELETE /inventory/{id} → Remove an item**
- **Implement validation for stock values.**

Expected Output:

• **API capable of managing inventory stock levels.**

Prompt:-

Create a **Warehouse Inventory REST API** using **Node.js and Express** to manage product stock. Each item should contain an ID, item name, and stock quantity stored in an in-memory list. Implement endpoints to display all inventory items, add new products, modify stock values, and delete items. Include validation to ensure that stock quantity cannot be negative.

Code:-

Task-2.js > ...

```
//Create a Warehouse Inventory REST API using Node.js and Express to manage product stock. Each item sh
const app = express();
const port = 3000;
app.use(express.json());

let inventory = [
    // Sample item for testing
    { id: 1, name: 'Item A', stock: 10 },
    { id: 2, name: 'Item B', stock: 20 },
    { id: 3, name: 'Item C', stock: 30 }
];
let nextId = 4;

// GET /inventory - List all items
app.get('/inventory', (req, res) => {
    res.json(inventory);
});
// POST /inventory - Add new item
app.post('/inventory', (req, res) => {
    const { name, stock } = req.body;
    if (typeof name !== 'string' || typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Name must be a string and stock must be a
    }
    const newItem = { id: nextId++, name, stock };
    inventory.push(newItem);
    res.status(201).json(newItem);
}
);
// PUT /inventory/:id - Update stock quantity
app.put('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { stock } = req.body;
    if (typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Stock must be a non-negative number.' });
    }
    const item = inventory.find(i => i.id === id);
    if (!item) {
        return res.status(404).json({ error: 'Item not found.' });
    }
    item.stock = stock;
    res.json(item);
});
// DELETE /inventory/:id - Remove item
app.delete('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
```

Task-2.js > ...

```javascript
app.delete('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = inventory.findIndex(i => i.id === id);
    if (index === -1) {
        return res.status(404).json({ error: 'Item not found.' });
    }
    const removedItem = inventory.splice(index, 1);
    res.json(removedItem[0]);
});
// Start the server
const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Server is running on port http://localhost:${PORT}`);
});
```

Output:-

Pretty-print

[{"id":1,"name":"Item A","stock":10},{"id":2,"name":"Item B","stock":20},{"id":3,"name":"Item C","stock":30}]

Justification:
The program builds a RESTful API that helps manage warehouse inventory records. It supports listing products, adding new inventory items, updating stock quantities, and removing items from the system. Validation checks prevent incorrect stock values and ensure data consistency. The API responses are structured in JSON format for easy integration with front-end systems.

Task 3 – Hotel Room Booking API
Task:
Develop a RESTful API for hotel room booking.
Instructions:
• Endpoints required:
o GET /rooms → View available rooms
o POST /rooms/book → Book a room
o GET /rooms/{id} → View booking details
o DELETE /rooms/{id} → Cancel booking
• Implement error handling for unavailable rooms.
Expected Output:
• API simulating hotel room booking operations.

Prompt:-

SR UNIVERSITY

Design a **Hotel Room Booking REST API** using **Node.js and Express** that manages room availability and bookings. Maintain a list of rooms containing an ID, room number, and booking status. Provide endpoints to view available rooms, book a room, retrieve booking details by ID, and cancel an existing booking. Include error messages if a room is already booked or does not exist.

Code:-

Task-3.js > ...

```
//Design a Hotel Room Booking REST API using Node.js and Express that manage
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let rooms = [
    // Sample rooms for testing
    { id: 1, roomNumber: '201', isBooked: false },
    { id: 2, roomNumber: '202', isBooked: false },
    { id: 3, roomNumber: '203', isBooked: false }
];
let nextId = 4;
// GET /rooms - List available rooms
app.get('/rooms', (req, res) => {
    const availableRooms = rooms.filter(room => !room.isBooked);
    res.json(availableRooms);
});
// POST /rooms/book - Book a specific room
app.post('/rooms/book', (req, res) => {
    const { roomNumber } = req.body;
    const room = rooms.find(r => r.roomNumber === roomNumber);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (room.isBooked) {
        return res.status(400).json({ error: 'Room is already booked.' });
    }
    room.isBooked = true;
    res.json({ message: `Room ${roomNumber} has been booked.`, room });
});
// GET /rooms/:id - View specific booking details
app.get('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    res.json(room);
});
```

```
/ DELETE /rooms/:id - Cancel a booking
pp.delete('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (!room.isBooked) {
        return res.status(400).json({ error: 'Room is not booked.' });
    }
    room.isBooked = false;
    res.json({ message: `Booking for room ${room.roomNumber} has been canceled.`, room });
);
/ Start the server
pp.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
```

Output:-

Justification:

This API simulates a hotel booking management system. It allows users to check available rooms, reserve a room, view booking details, and cancel reservations. The system includes validation to ensure that rooms cannot be booked twice and that operations are performed on valid room IDs. All responses are returned in JSON format for better communication with other applications.

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system.

Instructions:

• Endpoints required:

o GET /candidates → List candidates

o POST /vote → Cast a vote

o GET /results → View voting results

• Ensure one vote per user using in-memory validation.

Expected Output:

• API simulating a secure online voting process.

Prompt:-

Develop a **simple Online Voting REST API** using **Node.js and Express** that manages candidates and voting results. The system should allow users to view the list of candidates, submit a vote using a user ID and candidate ID, and check the current voting results. Implement an in-memory check to ensure that each user can vote only once.

Code:-

Task-4.js > ...

```javascript
//Develop a simple Online Voting REST API using Node.js and Express that manages candidate
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let candidates = [
    // Sample candidates for testing
    { id: 1, name: 'Candidate A', votes: 0 },
    { id: 2, name: 'Candidate B', votes: 0 },
    { id: 3, name: 'Candidate C', votes: 0 }
];
let votes = []; // To track user votes
// GET /candidates - List all candidates
app.get('/candidates', (req, res) => {
    res.json(candidates);
});
// POST /vote - Cast a vote
app.post('/vote', (req, res) => {
    const { userId, candidateId } = req.body;
    if (!userId || !candidateId) {
        return res.status(400).json({ error: 'User ID and Candidate ID are required.' });
    }
    if (votes.includes(userId)) {
        return res.status(400).json({ error: 'You have already voted.' });
    }
    const candidate = candidates.find(c => c.id === candidateId);
    if (!candidate) {
        return res.status(404).json({ error: 'Candidate not found.' });
    }
    candidate.votes += 1;
    votes.push(userId);
    res.json({ message: `Vote cast for ${candidate.name}.`, candidate });
});
// GET /results - Display current vote totals
app.get('/results', (req, res) => {
    res.json(candidates);
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-

localhost:3000/candidates

Problems - LeetCode

Pretty-print

[{"id":1,"name":"Candidate A","votes":0},{"id":2,"name":"Candidate B","votes":0},{"id":3,"name":"Candidate C","votes":0}]

Justification:

The API demonstrates a basic online voting mechanism using RESTful principles. It allows users to see available candidates, cast votes, and view the vote counts. To maintain fairness, the system checks whether a user has already voted before accepting another vote. This validation prevents duplicate voting and maintains the integrity of the voting process.

SR UNIVERSITY

Task 5 – Teacher Management API
Task:
Develop a RESTful API to manage teacher records.
Instructions:
• Endpoints required:
o GET /teachers → List all teachers
o POST /teachers → Add a new teacher
o PUT /teachers/{id} → Update teacher details
o DELETE /teachers/{id} → Remove a teacher
• Ensure proper error handling and JSON responses.
Expected Output:
• Functional API for managing teacher information.

Prompt:-
Create a **Teacher Management REST API** using **Node.js and Express** to perform CRUD operations on teacher records. Each record should include teacher ID, name, subject, and department. Implement endpoints to list all teachers, add new teacher details, update information by ID, and delete teacher records. Return appropriate JSON responses and error messages when a teacher is not found.

Code:-

Task-5.js > ...

```
//Create a Teacher Management REST API using Node.js and Express to perform CRUD operations on t
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let teachers = [
    // Sample teachers for testing
    { id: 1, name: 'John Doe', subject: 'Mathematics', department: 'Science' },
    { id: 2, name: 'Jane Smith', subject: 'English', department: 'Arts' },
    { id: 3, name: 'Emily Johnson', subject: 'History', department: 'Social Studies' }
];
let nextId = 4;
// GET /teachers - List all teachers
app.get('/teachers', (req, res) => {
    res.json(teachers);
});
// POST /teachers - Add new teacher
app.post('/teachers', (req, res) => {
    const { name, subject, department } = req.body;
    if (!name || !subject || !department) {
        return res.status(400).json({ error: 'Name, subject, and department are required.' });
    }                                          (property) subject: any
    const newTeacher = { id: nextId++, name, subject, department };
    teachers.push(newTeacher);
    res.status(201).json(newTeacher);
});
```

```
// PUT /teachers/:id - Update teacher details
app.put('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { name, subject, department } = req.body;
    const teacher = teachers.find(t => t.id === id);
    if (!teacher) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    if (name) teacher.name = name;
    if (subject) teacher.subject = subject;
    if (department) teacher.department = department;
    res.json(teacher);
});
// DELETE /teachers/:id - Remove a teacher
app.delete('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = teachers.findIndex(t => t.id === id);
    if (index === -1) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    teachers.splice(index, 1);
    res.json({ message: 'Teacher removed successfully.' });
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-

Justification:
This API provides a simple system for managing teacher information through RESTful services. It supports creating new teacher records, retrieving existing records, updating teacher information, and deleting records when necessary. Proper error handling ensures that invalid requests are managed effectively, and JSON responses allow smooth interaction with client applications.